

Herencia

Explicación completa sobre la **Herencia** en la Programación Orientada a Objetos (POO), uno de sus pilares fundamentales.

La **herencia** es un mecanismo que permite a una clase (llamada *clase hija* o *subclase*) adquirir (heredar) las propiedades y métodos de otra clase (llamada *clase padre* o *superclase*).

Piénsalo como la herencia genética 🧬. Tú heredas características de tus padres (color de ojos, tipo de sangre), pero también tienes tus propias características únicas.

Ventajas clave:

- **Reutilización de código:** No tienes que reescribir la lógica que ya existe en la clase padre.
- **Organización:** Crea una jerarquía lógica. Un "Omnibus" **ES UN** "Vehículo". Un "Perro" **ES UN** "Animal". Esta relación "ES UN" (IS A) es la señal de que debes usar herencia.

Conceptos Clave y Palabras Reservadas

1. **extends** (Clase Padre y Clase Hija)

Para que una clase herede de otra, usamos la palabra clave **extends**.

PHP

```
// --- CLASE PADRE (Superclase) ---  
// Contiene las propiedades y métodos comunes a todos los vehículos.  
class Vehiculo {  
    protected $marca;  
    protected $modelo;  
    protected $encendido = false;  
  
    public function __construct($marca, $modelo) {  
        $this->marca = $marca;  
        $this->modelo = $modelo;  
    }  
  
    public function arrancar() {  
        $this->encendido = true;  
        echo "El {$this->marca} {$this->modelo} ha arrancado.<br>";  
    }  
  
    public function apagar() {
```

```

        $this->encendido = false;
        echo "El {$this->marca} {$this->modelo} se ha apagado.<br>";
    }
}

// --- CLASE HIJA (Subclase) ---
// Hereda TODO lo de Vehiculo y añade su propia funcionalidad.
class Omnibus extends Vehiculo {

    // Esta es una propiedad específica de Omnibus
    public $capacidadPasajeros;

    public function __construct($marca, $modelo, $capacidad) {
        // Llamamos al constructor del PADRE para que asigne marca y modelo
        parent::__construct($marca, $modelo);

        // Asignamos la propiedad de la HIJA
        $this->capacidadPasajeros = $capacidad;
    }

    // Este es un método específico de Omnibus
    public function abrirPuertas() {
        echo "Abriendo puertas para {$this->capacidadPasajeros} pasajeros.<br>";
    }
}

// --- Uso ---
$miBus = new Omnibus('Mercedes-Benz', 'Citaro', 80);

// $miBus puede usar métodos de Vehiculo...
$miBus->arrancar(); // "El Mercedes-Benz Citaro ha arrancado."

// ...y también métodos propios de Omnibus.
$miBus->abrirPuertas(); // "Abriendo puertas para 80 pasajeros."

```

2. Visibilidad: **public**, **private** y **protected** 🔑

La visibilidad es crucial en la herencia:

- **public**: Accesible desde cualquier lugar (dentro de la clase, por las clases hijas y desde fuera).
- **private**: Solo accesible desde la clase que lo definió. **Ni siquiera las clases hijas pueden acceder a él.**
- **protected**: Accesible desde la clase que lo definió **y desde todas sus clases hijas**. No es accesible desde fuera.

Regla general: Si quieres que tus clases hijas puedan acceder y manipular una propiedad o método, pero quieres que el resto del mundo no pueda, usa `protected`. Por eso `$marca` y `$modelo` en `Vehiculo` son `protected`.

3. Sobrescritura de Métodos (Overriding)

Una clase hija puede "sobrescribir" un método de la clase padre para darle un comportamiento diferente.

Ejemplo: Queremos que el método `arrancar()` del `Omnibus` haga algo extra.

PHP

```
class Omnibus extends Vehiculo {

    // ... (constructor y otras propiedades) ...

    // Sobrescribimos el método arrancar()
    public function arrancar() {
        echo "Revisando sistema de frenos de aire...<br>";

        // Llamamos al método arrancar() ORIGINAL del padre
        parent::arrancar();

        echo "Sistema de tickets activado.<br>";
    }
}

$miBus = new Omnibus('Mercedes-Benz', 'Citaro', 80);
$miBus->arrancar();
```

Salida:

```
Revisando sistema de frenos de aire...
El Mercedes-Benz Citaro ha arrancado.
Sistema de tickets activado.
```

parent::

Como viste, `parent::` es una palabra clave especial que nos permite llamar a un método de la clase padre, incluso si lo hemos sobrescrito. Es fundamental para **extender** la funcionalidad sin tener que **repetir** el código.

4. Evitando la Herencia y la Sobrescritura: **final** ⚠

A veces no quieres que otros puedan heredar de tu clase o sobrescribir un método específico.

Clase **final:** No se puede heredar de ella.

PHP

```
final class ConfiguracionGlobal {  
    // ...  
}
```

// ESTO DARÁ UN ERROR FATAL:

```
// class MiConfig extends ConfiguracionGlobal {}
```

-

Método **final:** No se puede sobrescribir en una clase hija.

PHP

```
class Vehiculo {  
    // Hacemos que el método de seguridad no se pueda modificar  
    public final function obtenerNumeroChasis() {  
        // ... lógica para obtener el ID único  
    }  
}
```

```
class Omnibus extends Vehiculo {  
    // ESTO DARÁ UN ERROR FATAL:  
    // public function obtenerNumeroChasis() { ... }  
    }  
}
```

- }